

Linked list

LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Know about linked list
- Implement singly linked list, doubly linked list and circular linked list
- Know different applications of linked list

A linked list is a linear collection of data objects called nodes, each pointing to the next node. A data element and an address to the next node may be present in each node. This data structure makes it simple to add and remove data objects. Other data structures such as stacks and queues can be implemented using the linked list data structure.

The most essential advantage of a linked list over an array is that the memory elements used to hold the data components do not have to be contiguous, hence the size of the linked list is not restricted to the contiguous memory locations available. Linked lists also allow for the insertion and deletion of nodes at any point in the list with a fixed number of operations.

Linked lists make it difficult to access data. As a result, identifying a node with a certain data element or the last node in a linked list may necessitate sequential scanning of the majority or all of the elements.

4.1 SINGLY LINKED LISTS

Singly linked lists contain nodes which have two fields: a 'data' field and a 'next' field. The 'next' field contains the address of the next node. Sometimes the address is also called a pointer to the next node (Figure 4.1).



Figure 4.1 Node structure

We describe the node in the linked list as a class in Python with two attributes:

Python program 1

```
class Node:      # Definition of a Node
    def __init__(self, key):
        self.data = key
        self.next = None
```

This class definition creates an object of type Node. It has two attributes: data and next. Initially, the next is assigned to None and data is assigned to the key, which is provided while creating the object.

4.1.1 Singly linked lists: Creation

We first create the start node and keep on linking the new nodes. There is no limit to the number of nodes in the linked lists. The following Python program explains better the process of creation of the singly linked lists.

Program continues...

```
class SLinkedList:
# Class definition of a Single Linked List
    def __init__(self):
        self.start = Node(None)
```

The class SLinkedList defines a single linked list with an attribute *start*, which is an object of type node. The *start* is a pointer that points to the starting node of the linked list.

Program continues...

```
# Main Program
if __name__ == '__main__':
    list1 = SLinkedList()
    Ans = 'y'
    while (Ans == 'Y' or Ans == 'y'):
        x = input("Enter data to create the node :")
        NewNode = Node(x)
        if (list1.start == None):
            list1.start = NewNode
        else:
            temp = list1.start
            while(temp.next != None):
                temp = temp.next
            temp.next=NewNode
        Ans = input("Do you want to add More (Y | y) :")
```

```

print("The Elements in the Linked List are : ", end = "")
printval = list1.start
while printval is not None:
    print (printval.data ,end= " ")
    printval = printval.next
print("\n")

```

The main program creates an object of `SLinkedList()`, `list1`. When there is no element in the linked list, `start` points to *None*. When a new Node is created, the `start` points to the newly created node `list1.start = NewNode`. Otherwise, we take a variable *temp*, which initially points to the start node. If *temp* next is not *None*, move it to the next node. When such a node is found, assign the *NewNode* to the *temp.next*. Then print the elements in the linked list. This process continues as long as the value of *Ans* is “Y” or “y”.

A variable *printval* is assigned to the start node. As long as *printval* is not *None*, it will print the data part in it and then it is moved to the next node. Moving to next node is done by `printval = printval.next`.

A sample linked list

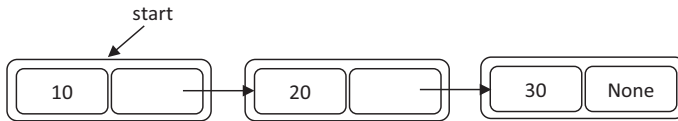


Figure 4.2 Linked List

Output

```

Enter data to create the node :10
The Elements in the Linked List are : None 10

Do you want to add More (Y | y) :y
Enter data to create the node :20
The Elements in the Linked List are : None 10 20

Do you want to add More (Y | y) :y
Enter data to create the node :30
The Elements in the Linked List are : None 10 20 30

Do you want to add More (Y | y) :n

```

4.1.2 Operations in a singly linked list

Operations that need to be performed on the linked lists are traversal, insertion and deletion.

i. Traversal

Traversal refers to visiting each node of a singly linked list. Before visiting each node in a linked list we must define the class of a node and class of a singly linked list.

Python program 2

```
class Node:      # Definition of a Node
    def __init__(self, key=None):
        self.data = key
        self.next = None

class SLinkedList: # Class definition of a Single Linked List
    def __init__(self):
        self.start = Node(None)
```

While printing the elements of a singly linked list, each node of the linked list must be visited. To do that we need to take a pointer *temp*, which is assigned to *start*. While *temp* is not *None*, print the data part in *temp* and move the pointer to the next node in the singly linked list.

Program continues...

```
def listprint(self): # Printing the elements of the list
    print("The Elements in the Linked List are : ", end = "")
    temp = self.start
    while temp is not None:
        print (temp.data ,end= " ")
        temp = temp.next
```

ii. Insertion

There are four positions where we can insert a node: in the front, at the end, after a given node and before a given node.

We have already created a linked list in the previous program with data elements 10, 20, 30.

1. Insertion at the front

Let us insert a node with some data element in the front. While inserting an element as a front node, there are two cases.

Case 1: When there is no element in the singly linked list, start points to *None*. After creating a new node, the start points to the new node.

Case 2: When there is a singly linked list present, the new node's next points to the start node of the already existing singly linked list. And then the existing start node moves to the new node (Figure 4.3).

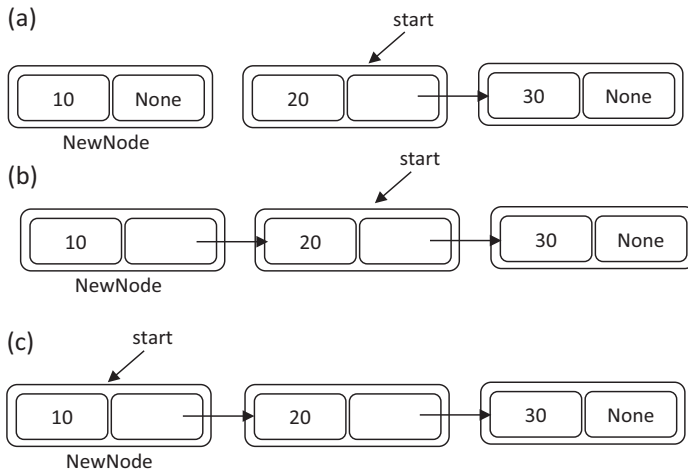


Figure 4.3 (a) Insertion at front. (b) Insertion at front. (c) Insertion at front

The Python program that does this is as follows:

Program continues...

```
def insert_At_Begining(self, newdata):
    NewNode = Node(newdata)
    if (self.start == None):
        self.start = NewNode
    else:
        NewNode.next = self.start
        self.start = NewNode
```

2. Insertion at the end

While inserting an element as a last node, there are two cases.

Case 1: When there is no element in the singly linked list, start points to *None*. After creating a new node, the start points to the new node.

Case 2: When there is a singly linked list present, assign a new pointer temp to start, move temp to the last node. Assign the next of temp to the new node (Figure 4.4).

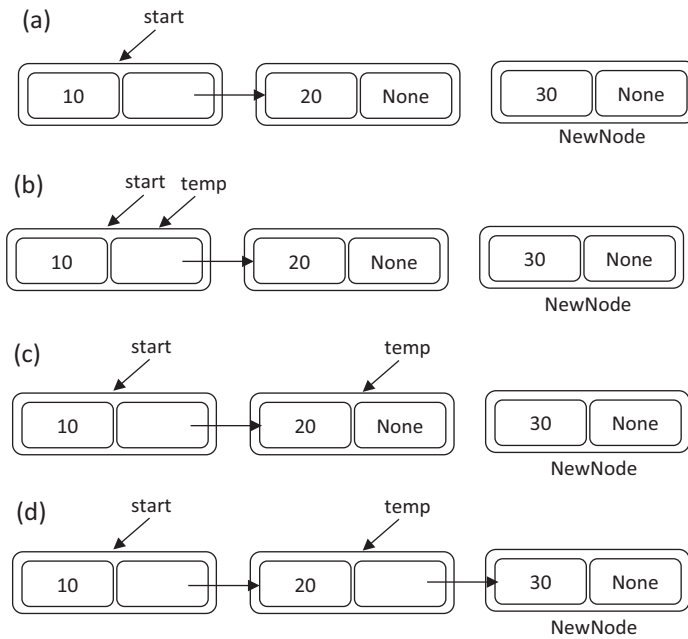


Figure 4.4 (a) Insertion at end. (b) Insertion at end. (c) Insertion at end. (d) Insertion at end

The function to do this can be expressed in the Python program as follows:

Program continues...

```
def insert_At_End(self,newdata):
    NewNode = Node(newdata)
    if (self.start == None):
        self.start = NewNode
    else:
        temp = self.start
        while(temp.next):
            temp = temp.next
        temp.next=NewNode
```

3. Insertion after a given node

Insertion in the middle is slightly more complicated since the link between two nodes is to be “broken” and reattached to the new node appropriately.

While inserting a node after a node with a given key, first, take a pointer *temp* and move the pointer to the location after which a new node to be inserted (Figure 4.5). There are two cases.

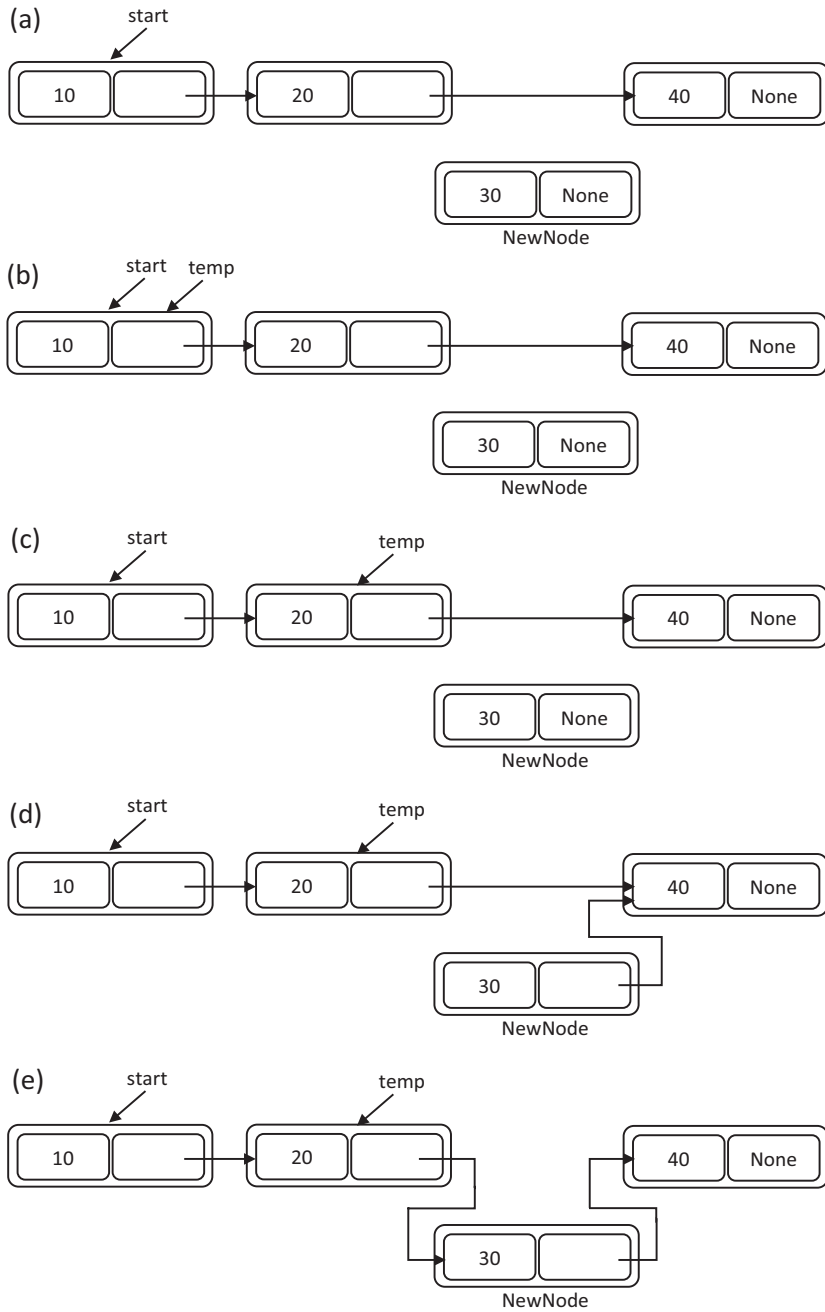


Figure 4.5 (a) New node created. (b) Assign temp to start node. (c) temp moved to the required node after which new node to be inserted. (d) Assign new node's next to temp's next node. (e) Assign temp's next to NewNode

Case 1: When the node with the given key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, then make the new node's next point to the node to which temp's next is pointing and then make the temp's next point to the new node.

Let us write a method in Python which does this.

Program continues...

```
def insert_After(self, key, newdata):
    temp = self.start
    while(temp.data != key):
        temp = temp.next
    if (temp.data != data and temp.next == None ):
        print(data, " Not Found")
    else:
        NewNode = Node(newdata)
        NewNode.next = temp.next
        temp.next = NewNode
```

4. Insertion before a given node

While inserting a node before a node with a given key, assign *temp* to the *start*. If the data in the first node is the same as the key (before which new node to be inserted), use the *insert_At_Begining()* method.

If the node to be inserted is not the first node then take a pointer *temp*, which points to the first node while checking the data in the second node (that is temp's next data). If it matches with the key, insert the new node as a second node, otherwise move the *temp* pointer to the next one. There are two cases.

Case 1: If the key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, then make the new node's next point to the node which temp's next is pointing to and then make the temp's next point to the new node.

The diagram is same as the previous one, except for the pointer.

Program continues...

```
def insert_Before(self, data,newdata):
    temp = self.start
    if (temp.data == data):
        self.insert_At_Begining(newdata)
    else:
        while(temp.next.data != data and temp.next !=
            None):
            temp = temp.next
        if (temp.next == None ):
            print(data, " Not Found")
        else:
            NewNode = Node(newdata)
            NewNode.next = temp.next
            temp.next = NewNode
```

iii. Deletion

Deletions from a linked list are comparatively easier than insertions. There are five types of deletions: deletion of the node in the front, deletion of the node at the end, deletion of a given node, deletion of the node after a given node and deletion of the node before a given node.

1. Deletion of the node at the front

Deleting a node that is the first node of a linked list is quite simple. Take a pointer *temp* and assign *temp* to the *start*. Move the start to its next node. Now delete the node pointed by *temp* (Figure 4.6).

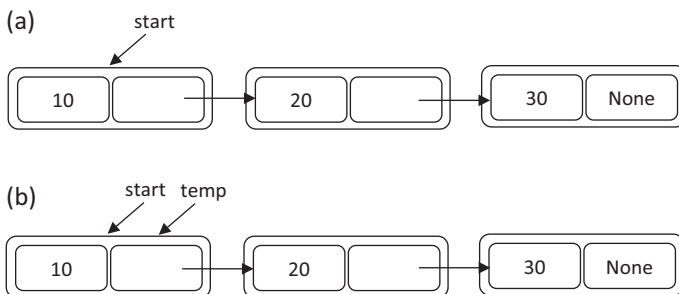


Figure 4.6 (a) Deletion of the node at the front. (b) Assign temp to start. (c) Move start to temp's next. (d) delete the node pointed by temp

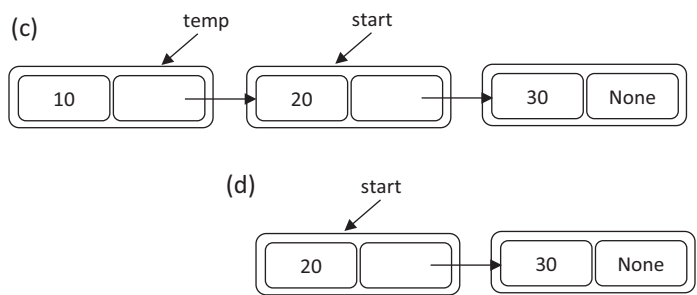


Figure 4.6 Continued

Program continues...

```
def del_first(self):
    temp = self.start
    self.start = temp.next
    del temp
```

2. Deletion of the node at the end

Deleting a node that is the last node of a linked list is quite simple. Take a pointer *temp* and assign *temp* to the *start* (Figure 4.7). There are two cases of deleting the last node.

Case 1: If the last node is the same as the first node. In other words, there is only one node present in the singlylinked list. Call the *del_first()* method.

Case 2: If there is more than one element in the linked list, move the *temp* pointer to the last but one node using a while loop. Assign one more pointer *temp1* to *temp*'s next. Assign *temp*'s next to *None* and delete *temp1*.

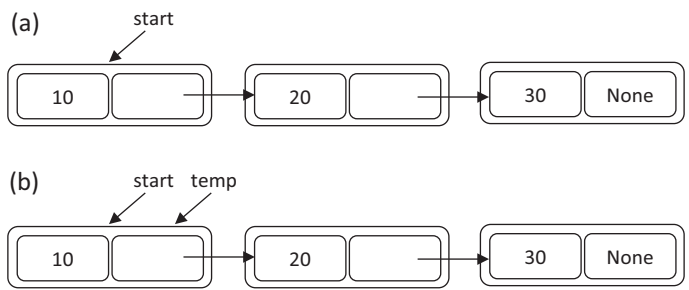


Figure 4.7 (a) Deletion of the node at the end. (b) Assign pointer *temp* to *start*. (c) move *temp* to last but one node. (d) Assign one more pointer *temp1* to *temp*'s next. (e) Assign *None* to *temp*'s next (f) Delete *temp1*

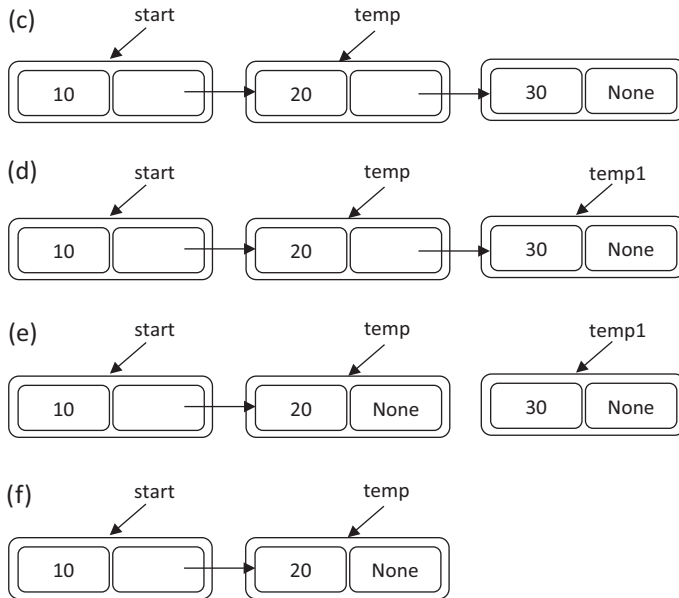


Figure 4.7 Continued

Program continues...

```
def del_last(self):
    temp = self.start
    if temp.next == None :
        self.del_first()
        return
    while(temp.next.next != None):
        temp = temp.next
    temp1 = temp.next
    temp.next = None
    del temp1
```

3. Deletion of the node

Deleting a node from the middle is a little more complicated since we have to first traverse the list up to the node we wish to delete. Let the node to be deleted contain the key. Depending on the presence of the key there are three cases.

Case 1: If the key is present in the first node, call the *del_first()* method.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take two pointers *temp* and *temp1*. Assign *temp* to the *start* pointer and *temp1* to next of *temp*. Keep moving both the pointers *temp* and *temp1*, till *temp1*'s data matches with the *key*, then assign *temp*'s next to *temp1*'s next and delete *temp1* (Figure 4.8).

Case 3: If the key does not match with data of any node, display appropriate messages.

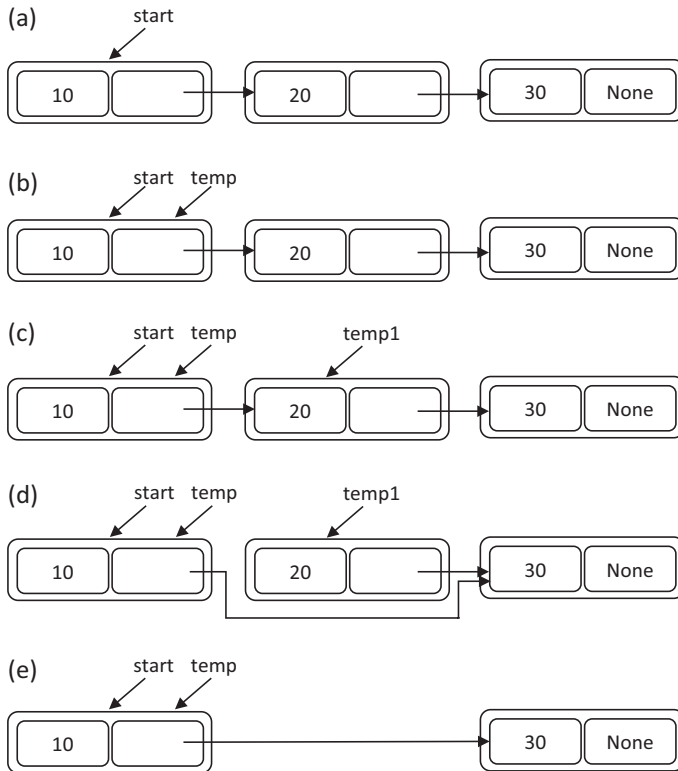


Figure 4.8 (a) Deletion of the node. (b) Assign pointer *temp* to *start*. (c) move *temp* to node previous to the node to be deleted. (d) Assign one more pointer *temp1* to *temp*'s next. (e) Assign *temp*'s next to *temp1*'s next (f) Delete *temp1*

The following code deletes the node containing the key.

Program continues...

```
def del_node(self, key):
    temp = self.start
    if (temp.data == key ):
        self.del_first()
    return
```

```

temp1 = temp.next
while(temp1.data != key):
    temp = temp.next
    temp1 = temp1.next
if (temp1.data != key and temp1.next == None ):
    print(data, " Not Found")
else:
    temp.next = temp1.next
    del temp1

```

4. Deletion of the node after a given node

Depending on the presence of the key, there are three cases.

Case 1: If there is only node and the key matches with the data present in the node. As there is no node after the first node, display appropriate messages.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp* and assign it to the *start* pointer. Keep moving the pointer until the temp's data matches with the *key*. Take one more pointer *temp1* and assign it to temp's next. Now assign temp's next to temp1's next and delete *temp1*.

Case 3: If the key does not match with data of any node, display appropriate messages.

The diagram is the same as the previous one.

Program continues...

```

def del_after(self, data):
    temp = self.start
    if (temp.data == data and temp.next == None ):
        print("There is only one element in the list...")
        print("Next element can not be deleted..")
        return
    while(temp.data != data and temp.next !=None):
        temp = temp.next
    if (temp.data != data and temp.next == None ):
        print(data, " data Not found")
    elif (temp.data == data and temp.next == None ):
        print(data, "is the Last node. Next, node cannot be deleted")
    else:
        temp1 = temp.next
        temp.next = temp1.next
        del temp1

```

5. Deletion of the node before a given node

Depending on the presence of the key, there are four cases.

Case 1: If the key matches with the data present in the first node. As there is no node before the first node, before the node cannot be deleted, display appropriate messages.

Case 2: If the key is in the second node, call the *del_first()* method.

Case 3: If the key is not the second node, it can be anywhere in the linked list. To traverse the linked list, take two pointers *temp* and *temp1*. Assign *temp* to the *start* pointer and *temp1* to next-next of *temp*. Keep moving both the pointers *temp* and *temp1*, until *temp1*'s data matches with the *key*. Take one more pointer *temp2* and assign *temp*'s next. For deleting, assign *temp*'s next to *temp2*'s next and delete *temp2* (Figure 4.9).

Case 4: If the key does not match with data of any node, display appropriate messages.

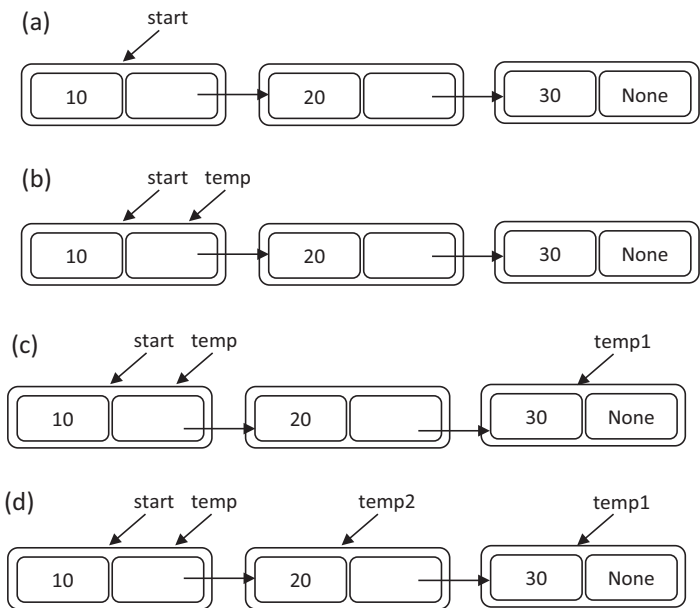


Figure 4.9 (a) Deletion of the node. (b) Assign temp to start. (c) Assign temp1 to last node. (d) Assign temp2 to last but 1 node (f) connect next of temp to next of temp2 (e) Deletion temp2

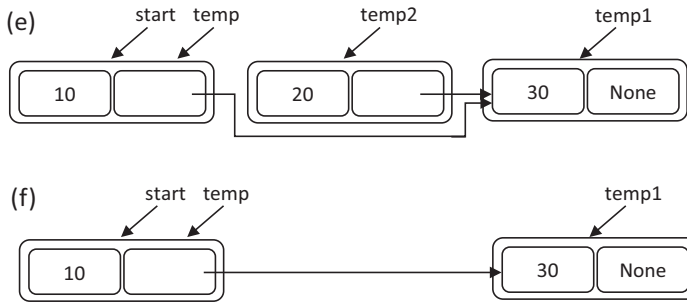


Figure 4.9 Continued

Program continues...

```
def del_before(self, data):
    temp = self.start
    if (temp.data == data ):
        print("There is only one element in the list...")
        print("Previous element cannot be deleted..")
        return
    if(temp.next.data == data):
        self.del_first()
        return
    temp1 = temp.next.next
    while(temp1.data != data and temp1.next !=None):
        temp1 = temp1.next
        temp = temp.next

    if (temp1.data != data and temp1.next == None ):
        print(data, " data Not found")
    else:
        temp2 = temp.next
        temp.next = temp2.next
        del temp2
```

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Create a Single Linked List ")
    print("2. Insert at the Beginneing ")
    print("3. Insert at the End")
    print("4. Insert After a given Data ")
    print("5. Insert before a given Data")
```

```
print("6. Delete the First node")
print("7. Delete the Last node")
print("8. Delete the a given node")
print("9. Delete After a given Data")
print("10. Delete before a given Data")
print("11. Exit ")
opt = int(input("Enter a valid menu item ... "))
return opt

# Main Program
if __name__ == '__main__':
    list1 = SLinkedList()
    i=1
    while (i > 0 and i <=10 ):
        i = menu()
        if i == 1:
            x = input("Enter data to create the first node :")
            list1.start = Node(x)
            list1.listprint()

        elif (i==2):
            x = input("Enter data to create the node :")
            list1.insert_At_Begining(x)
            list1.listprint()

        elif (i==3):
            x = input("\n Enter data to create the node :")
            list1.insert_At_End(x)
            list1.listprint()

        elif (i==4):
            x = input("\n Enter data to create the node :")
            y = input("Node to be created after data :")
            list1.insert_After(y,x)
            list1.listprint()

        elif (i==5):
            x = input("\n Enter data to create the node :")
            y = input("Node to be created before data :")
            list1.insert_Before(y,x)
            list1.listprint()

        elif (i==6):
            list1.del_first()
            list1.listprint()

        elif (i==7):
            list1.del_last()
            list1.listprint()
```

```

elif (i==8):
    x = input("Node to be deleted :")
    list1.del_node(x)
    list1.listprint()

elif (i==9):
    x = input("Node to be deleted after data :")
    list1.del_after(x)
    list1.listprint()

elif (i==10):
    x = input("Node to be deleted before data :")
    list1.del_before(x)
    list1.listprint()

else:
    print("End")

```

Output

```

~~~ MENU ~~~
1. Create a Single Linked List
2. Insert at the Beginning
3. Insert at the End
4. Insert After a given Data
5. Insert before a given Data
6. Delete the First node
7. Delete the Last node
8. Delete the a given node
9. Delete After a given Data
10. Delete before a given Data
11. Exit
Enter a valid menu item ... 1
Enter data to create the first node :10
The Elements in the Linked List are : 10

```

In some implementations, an extra dummy node, usually called a *sentinel* node is added before the first data record and after the last data record. Every list, therefore, has two sentinel nodes, the first one and the last one, even when the list is empty. Since the reference to the first node gives access to the whole list, it is often called the “address” or “pointer”, or handle to the list.

4.2 DOUBLY LINKED LISTS

In doubly linked lists each node, besides containing a pointer to the next node, also contains a pointer to the previous node (Figure 4.10).

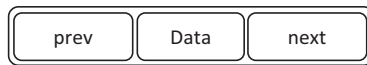


Figure 4.10 Node in a double linked list

A doubly linked list contains an extra pointer, usually called prev (previous), together with the next pointer and data (Figure 4.11).

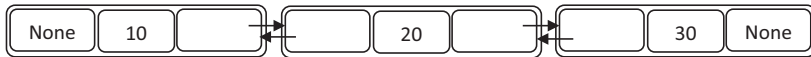


Figure 4.11 Structure of a doubly linked list

A node in a doubly linked list can be represented as:

Python program 3

```
class Node: # Class Definition of a Node in Double Linked List
    def __init__(self, dataval=None):
        self.prev = None
        self.data = dataval
        self.next = None

class DLinkedList: # Class Definition of a Double Linked List
    def __init__(self):
        self.start = Node(None)

    def listprint(self): #Printing the elements of a DLL List
        print("The Elements in the Linked List are : ", end = "")
        printval = self.start
        while printval is not None:
            print (printval.data ,end= " ")
            printval = printval.next
        print("\n")
```

4.2.1 Operations in a doubly linked list

Operations that need to be performed on doubly linked lists are traversal, inserting and deleting.

i. Traversal

Traversal refers to visiting each node in a data structure. Readers are advised to refer the traversal in singly linked list.

ii. Insertion

A node can be inserted in a doubly linked list in four different ways: at the front, at the end, after a given node and before a given node.

1. Insertion of a node at the front

When a node is added in the front, the newly added node becomes the *start* node.

Let us insert a node with some data element in the front. While inserting an element as a front node, there are two cases:

Case 1: When there is no element in the doubly linked list, the *start* points to *None*. After creating a new node with *prev* and *next* set to *None*, the *start* points to the new node.

Case 2: When there is a doubly linked list, follow these three steps to perform the operation (Figure 4.12).

- i. Set the *next* pointer of the new node to *start*.
- ii. Make the *prev* of the *start* node point to the new node
- iii. Move the *start* to the new node.

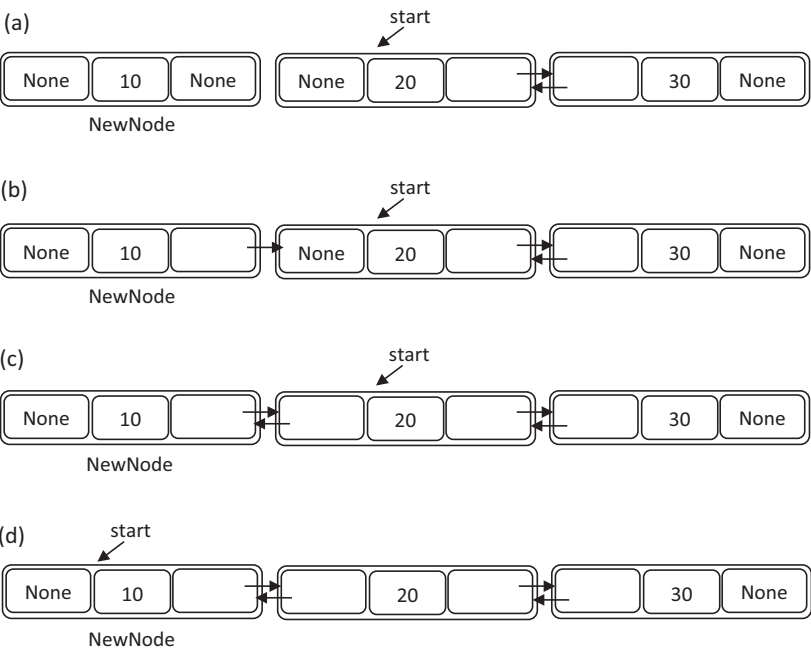


Figure 4.12 (a) create a new node (b) Assign next of new node to start (c) Assign previous of start to newnode (d) Assign start to new node

These steps can be converted to a Python program as follows:

Program continues...

```
def insert_At_Begining(self,newdata):
    NewNode = Node(newdata)
    if (self.start==None):
        self.start = NewNode
    else:
        NewNode.next = self.start
        self.start.prev = NewNode
        self.start = NewNode
```

2. Insert a node at the end

Insertion of a node as the last node in a double linked list is almost the same as insertion of a node as the last node in a single list. It also has two cases:

Case 1: When there is no element in the single linked list, start points to *None*. Call the `insert_At_Begining ()` method.

Case 2: When there is a doubly linked list present, assign a pointer *temp* to the *start*. Move *temp* to the node where next of the node is *None* (i.e., the last node). Assign *temp*'s *next* to New Node and then assign new node's *prev* to *temp* (Figure 4.13).

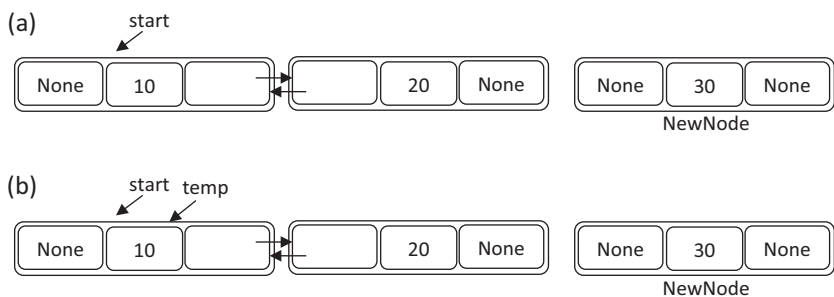


Figure 4.13 (a) Create a NewNode (b) Assign temp to start (c) Move temp till last node. (d) Assign next of temp to NewNode (e) Assign previous of NewNode to temp (e) Insertion of a node at the end in a doubly linked list

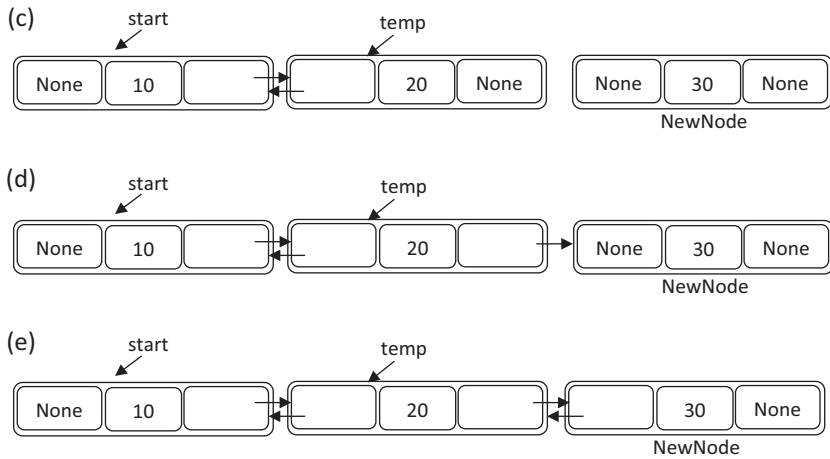


Figure 4.13 Continued

Program continues...

```
def insert_At_End(self,newdata):
    if (self.start == None):
        self.insert_At_Begining(newdata)
        return
    NewNode = Node(newdata)
    temp = self.start
    while(temp.next != None):
        temp = temp.next
    temp.next=NewNode
    NewNode.prev = temp
```

3. Insert a node after a given node in a doubly linked list

Insertion in the middle is slightly more complicated since the link between two nodes is to be “broken” and reattached to the new node appropriately.

While inserting a node after a node with a given key, first, take a pointer `temp` and move the pointer to the location after which a new node to be inserted. There are two cases:

Case 1: When the node with the given key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, check if temp's next points to *None*. If so, insert the new node as last node by calling *insert_At_End()* method, otherwise follow these four steps (Figure 4.14):

- i. Set a new node's next point to the node to which temp's next is pointing.
- ii. Assign the new node's prev to temp.
- iii. Make the temp's next-prev point to the new node.
- iv. Set temp's next to the new node.

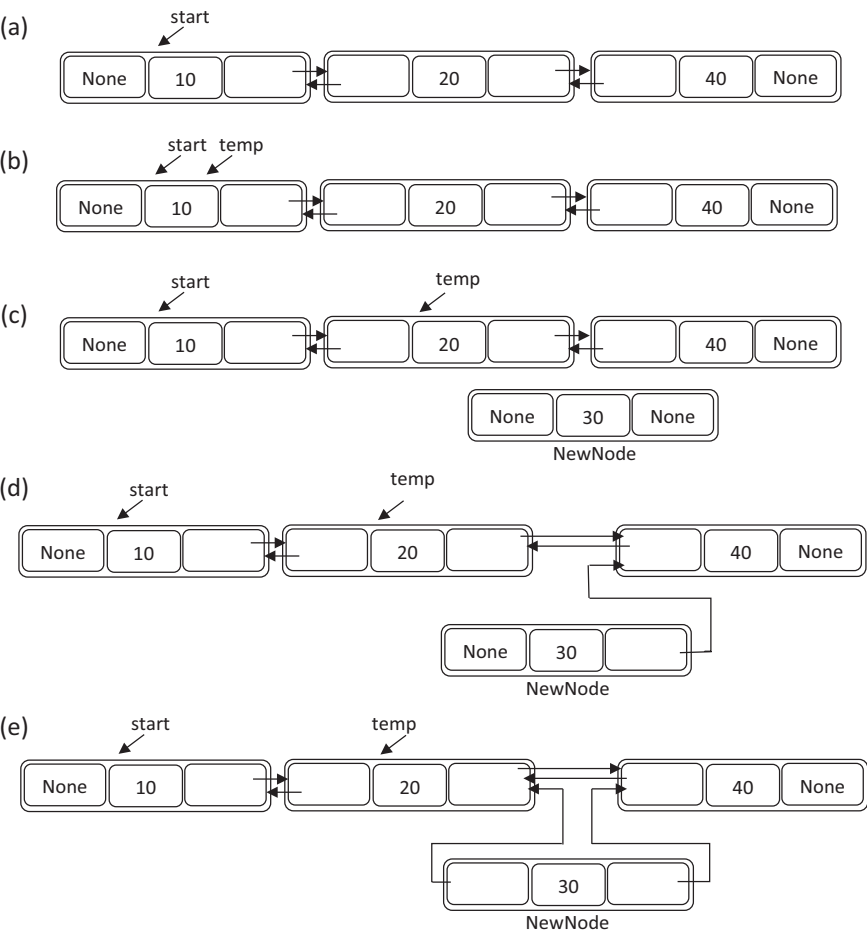


Figure 4.14 (a) Insert a node after a given node in a doubly linked list. (b) Assign Temp to start node (c) Move temp till required location (d) Assign newNode next to temp Next (e) Assign newnode previous to temp (f) Assign temp's next's previous to NewNode. (g) Assign temp's next to NewNode

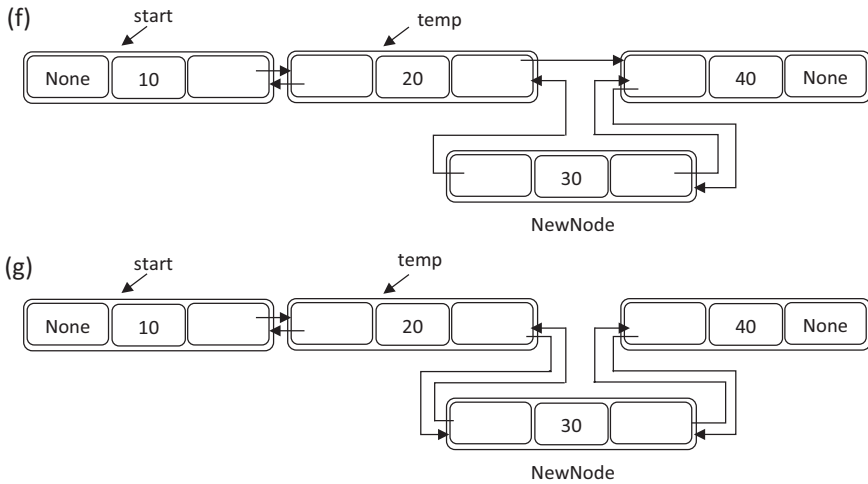


Figure 4.14 Continued

These steps can be executed through the *insert_After()* method shown next.

Program continues...

```
def insert_After(self, key, newdata):
    temp = self.start
    while(temp.data != key and temp.next != None):
        temp = temp.next
    if (temp.data != key and temp.next == None ):
        print(key, " Not Found")

    elif (temp.data == key and temp.next == None):
        self.insert_At_End(newdata)
    else:
        NewNode = Node(newdata)
        NewNode.next = temp.next
        NewNode.prev = temp
        temp.next.prev = NewNode
        temp.next = NewNode
```

4. Insert a node before a given node in a doubly linked list

While inserting a node before a node with a given key, assign *temp* to the *start*. If the data in the temp node is the same as the key (before which new node to be inserted), call *insert_At_Begining()* method.

If the node to be inserted is not the first node then make a pointer *temp* to check the data in the second node (that is *temp*'s next-data). If it matches with the key, insert the new node as a second node, otherwise move the *temp* pointer to the next one. There are two cases:

Case 1: If the key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, then make the new node's next point to the node that the *temp*'s next is pointing to and then follow these steps.

- i. Set a new node's next point to the node to which *temp*'s next is pointing.
- ii. Assign the new node's prev to *temp*.
- iii. Make the *temp*'s next-prev point to the new node.
- iv. Set *temp*'s next to the new node.

The diagram is same as the previous one, except for the pointer.

Program continues...

```
def insert_Before(self, key, newdata):
    temp = self.start
    if (temp.data == key):
        self.insert_At_Begining(newdata)
    else:
        while(temp.data != key and temp.next != None):
            temp = temp.next
        if (temp.next == None ):
            print(key, " Not Found")
        else:
            NewNode = Node(newdata)
            NewNode.next = temp.next
            NewNode.prev = temp
            temp.next.prev = NewNode
            temp.next = NewNode
```

iii. Deletion

Deletions from a linked list are comparatively easier than insertions. Once again there are five types of deletions: deletion of the first node, deletion of the node in the end, deletion of a given node, deletion of the node after a given node and deletion of the node before a given node.

1. Deletion of first node

Deleting a node which is the first node of a doubly linked list is quite simple. Take a pointer *temp* and assign it to the *start*. Move the *start* to its next node. Set the *start*'s *prev* to *None* and delete the node pointed by *temp*.

The diagram is almost the same as the deletion of the first node in the singly linked list.

Program continues...

```
def del_first(self):
    if(self.start != None):
        temp = self.start
        self.start = self.start.next
        temp.next = None
        del temp
    if (self.start != None ):
        self.start.prev=None
```

2. Deletion of the node at the end

Deleting a node that is the last node of a doubly linked list is quite simple. Take a pointer *temp* and assign *temp* to the *start*. There are two cases when deleting the last node.

Case 1: If the last node is the same as the first node. In other words, there is only one node present in the doubly linked list. Call the *del_first()* method.

Case 2: If there are more than one element in the doubly linked list, move the *temp* pointer to the next to last one node using a while loop. Assign one more pointer *temp1*, pointing to *temp*'s next. Assign *temp*'s next to *None* and delete *temp1*.

The diagram is almost the same as deletion of the first node in the singly linked list.

Program continues...

```
def del_last(self):
    temp = self.start
    if (temp.next == None) :
        self.del_first()
        return
    while(temp.next.next != None):
        temp = temp.next
    temp1 = temp.next
    temp.next = None
    del temp1
```

3. Deletion of a given node

Deleting a node from the middle is a little more complicated since we have to first traverse the list up to the node we wish to delete. Let the node to be deleted contain the key. Depending on the presence of the key, there are three cases:

Case 1: If the key is present in the first node, call the *del_first()* method.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp* and assign it to the *start* pointer. Keep moving the *temp* pointer until *temp*'s data matches with the *key*.

- i. If the key is not found, print the appropriate message.
- ii. If the key is found at the last node, call the *del_last()* method.
- iii. If the key is found in between, take one more pointer *temp1*, pointing to *temp*'s prev, assign *temp*'s next to *temp1*'s next, assign *temp1* to *temp*'s next-prev. Now delete *temp*.

Program continues...

```
def del_node(self, key):
    temp = self.start
    if (temp.data == key ):
        self.del_first()
        return
    while(temp.data != key and temp.next != None):
        temp = temp.next
    if (temp.data != key and temp.next == None ):
        print(data, " Not Found")
    elif(temp.data == key and temp.next == None):
        self.del_last()
    else:
        temp1=temp.prev
        print(temp1.data)
        temp1.next = temp.next
        temp.next.prev = temp1
        del temp
```

4. Deletion of the node after a given node

Depending on the presence of the key, there are two cases.

Case 1: If there is only node and the key matches with the data present in the node. As there is no node after the first node, display appropriate messages.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp* and assign it to the *start* pointer. Keep moving the pointer until *temp*'s data matches with the *key*.

- i. If the key does not match with data of any node, display appropriate messages.
- ii. If the key matches with the last node, call the *del_last()* method.
- iii. If the key matches with any other intermediate node, take one more pointer *temp1* and assign it to *temp*'s next. Assign *temp1*'s next-prev to *temp*, assign *temp*'s next to *temp1*'s next. Now delete the *temp1* node.

Program continues...

```
def del_after(self, key):
    temp = self.start
    if (temp.data == key and temp.next == None):
        print("There is only one element in the list...")
        print("Next element can not be deleted..")
        return
    while(temp.data != key and temp.next !=None):
        temp = temp.next

    if (temp.data != key and temp.next == None):
        print(key, " data Not found")

    elif (temp.data == key and temp.next == None):
        print(key, " is the Last node. Next, node cannot
        be deleted")

    elif(temp.data == data and temp.next.next == None):
        self.del_last()
    else:
        temp1=temp.next
        temp1.next.prev=temp
        temp.next = temp1.next
        del temp1
```

5. Deletion of the node before a given node

Depending on the presence of the key there are three cases.

Case 1: If the key matches with the data present in the first node. As there is no node before the first node, before the node cannot be deleted, display appropriate messages.

Case 2: If the key is in the second node, call the *del_first()* method.

Case 3: If the key is not the second node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp*. Assign *temp* to the *start* pointer. Keep moving *temp* pointers, until *temp*'s data matches with the *key*. There can be three options:

- i. If the key is not found, display appropriate message.
- ii. If *temp* is at the second node (*temp*'s *prev*-*prev* is *None*), call the *del_First()* method.
- iii. If *temp* is pointing to any other node, take one more pointer *temp1* and set it to *temp*'s *prev*.

Program continues...

```
def del_before(self, key):
    temp = self.start
    if (temp.data == key ):
        print("This is the first element in the list...")
        print("Previous element cannot be deleted..")
        return

    while(temp.data != key and temp.next !=None):
        temp = temp.next

    if (temp.data != key and temp.next == None ):
        print(data, " data Not found")

    elif(temp.data == key and temp.prev.prev == None):
        self.del_first()
    else:
        temp1=temp.prev
        temp1.prev.next = temp
        temp.prev = temp1.prev
        del temp1
```

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Create a Double Linked List ")
    print("2. Insert at the Beginneing ")
    print("3. Insert at the End")
    print("4. Insert After a given Data ")
    print("5. Insert before a given Data")
    print("6. Delete the First node")
    print("7. Delete the Last node")
    print("8. Delete the a given node")
    print("9. Delete After a given Data")
    print("10. Delete before a given Data")
    print("11. Exit ")
```

```
opt = int(input("Enter a valid menu item ... "))
return opt

if __name__ == '__main__':
    list1 = DLinkedList()
    i=1
    while (i > 0 and i <=10 ):
        i = menu()
        if i == 1:
            x = input("Enter data to create the first node :")
            list1.start = Node(x)
            list1.listprint()

        elif (i==2):
            x = input("Enter data to create the node :")
            list1.insert_At_Begining(x)
            list1.listprint()

        elif (i==3):
            x = input("\n Enter data to create the node :")
            list1.insert_At_End(x)
            list1.listprint()

        elif (i==4):
            x = input("\n Enter data to create the node :")
            y = input("Node to be created after data :")
            list1.insert_After(y,x)
            list1.listprint()

        elif (i==5):
            x = input("\n Enter data to create the node :")
            y = input("Node to be created before data :")
            list1.insert_Before(y,x)
            list1.listprint()

        elif (i==6):
            list1.del_first()
            list1.listprint()

        elif (i==7):
            list1.del_last()
            list1.listprint()

        elif (i==8):
            x = input("Node to be deleted :")
            list1.del_node(x)
            list1.listprint()

        elif (i==9):
            x = input("Node to be deleted after data :")
            list1.del_after(x)
            list1.listprint()
```

```
elif (i==10):
    x = input("Node to be deleted before data :")
    list1.del_before(x)
    list1.listprint()
else:
    print("End")
```

Advantages of doubly linked lists

1. The doubly linked list can be traversed in both directions.
2. We can access all the nodes starting from any node, not necessarily starting from the start.

Disadvantages of doubly linked lists

1. Every node in the doubly linked list needs extra space.
2. All operations need to maintain two pointers.

4.3 CIRCULAR LINKED LISTS

In the last node of a singly linked list, we have a *None* reference to indicate the lack of further nodes. If we make it point to the first node (or the *start* node) of the list we have a *circularly linked list*. In other words, the last element contains the address of the first element. We take one more pointer, *end*, to point to the last node on a circular linked list (Figure 4.15).

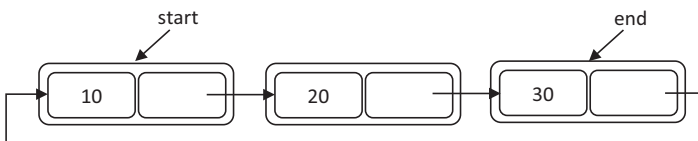


Figure 4.15 Circular linked lists

The node structure of a circular linked list and node structure of a singly linked list are the same. When we create a circular linked list, we take an additional point to point to the last node.

Python program 4

```
class Node:
    def __init__(self, dataval=None):
        self.data = dataval
        self.next = None
```

```

class CLinkedList:
    def __init__(self):
        self.start = Node(None)
        self.end = Node(None)
        self.start.next = self.end
        self.end.next = self.start

```

4.3.1 Operations in a circular linked list

There are three operations that can be performed on a circular list: look-up, insertion and deletion.

i. Look-up

Let us say, we are looking for data item x in the circular list. We execute the following steps:

1. Start at the head node. (The head node can be any one of the nodes.)
2. Follow the links from the head node until we reach the node containing the desired value, if it exists.
3. If we reach back to the head node, report failure.

Let us first create a circular list. If we are creating the first node in the list, let us call it head. We define the node as a structure:

Program continues...

```

def listprint(self):
    print("The Elements in the Linked List are : ", end = "")
    temp = self.start
    if (self.start == None):
        print("There is no data..")
        return
    else:
        while(True):
            print(temp.data, end= " ")
            temp = temp.next
            if (temp == self.start):
                return

```

ii. Insertion

Insertion can be at the beginning, at the end, after a given node and before a given node.

Even though there is no beginning or end in a circular list, we identify one of the nodes as the head node, and the node pointing to the head node as the last node.

1. Insertion at the beginning

When a node is added in the front, the newly added node becomes the *start* node.

Let us insert a node with some data element in the front. While inserting an element as a front node, there are two cases:

Case 1: When there is no element in the circular linked list, the *start* and *end* points to *None*. After creating a new node with *next* of the new nodes pointing to itself, the *start* and *end* points to the new node (Figure 4.16).

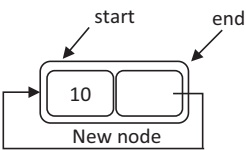


Figure 4.16 New node in circular linked lists

Case 2: When there is a circular linked list, follow these four steps to perform the operation (Figure 4.17).

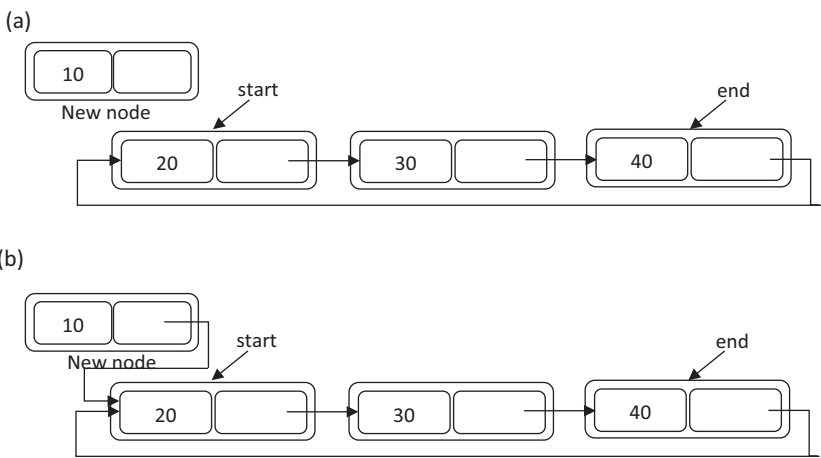


Figure 4.17 (a) Insert a node after a given node in a doubly linked list. (b) Assign NewNode's next to start. (c) Assign end's next to Newnode (d) Move start to Newnode

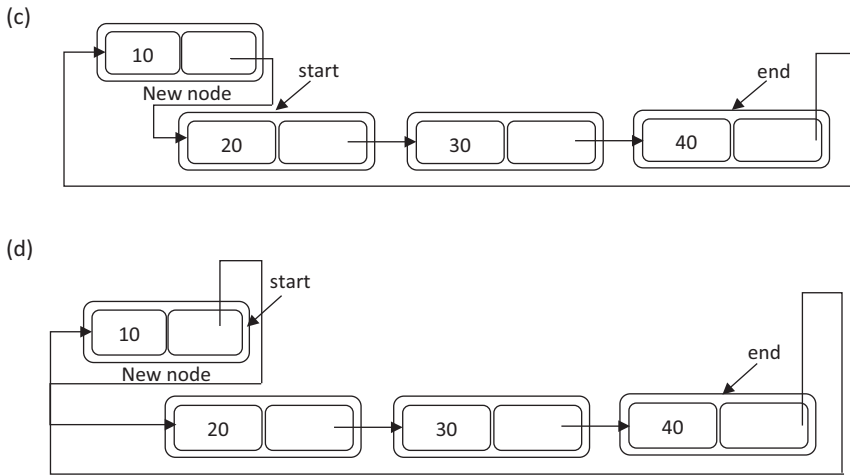


Figure 4.17 Continued

- i. Create a new node.
- ii. Set the *next* pointer of the new node to start.
- iii. Make the *next* of the *end* point to the new node.
- iv. Make a *start* point to the new node.

We execute the following Python code to insert a node with data item 10 in the beginning of the circular list with the variables declared earlier. The new node contains the data element 10.

Program continues...

```
def insert_At_Begining(self,newdata):
    NewNode = Node(newdata)
    if (self.start==None):
        self.start = NewNode
        NewNode=self.start
        self.end = self.start
        self.end.next = self.start
    else:
        NewNode.next = self.start
        self.start = NewNode
        self.end.next = self.start
```

2. Insertion at the end

When a node is added at the end, the newly added node becomes the *end* node.

Let us insert a node with some data element at the rear (Figure 4.18). While inserting an element as an end node, there are two cases:

Case 1: If the circular linked list is empty, then we simply add the new node as in case 1 of insertion at the beginning.

Case 2: When there is a circular linked list, follow these three steps to perform the operations (Figure 4.18).

1. Create a new node.
2. Point the end to the new node.
3. Point the end of the new node to start.

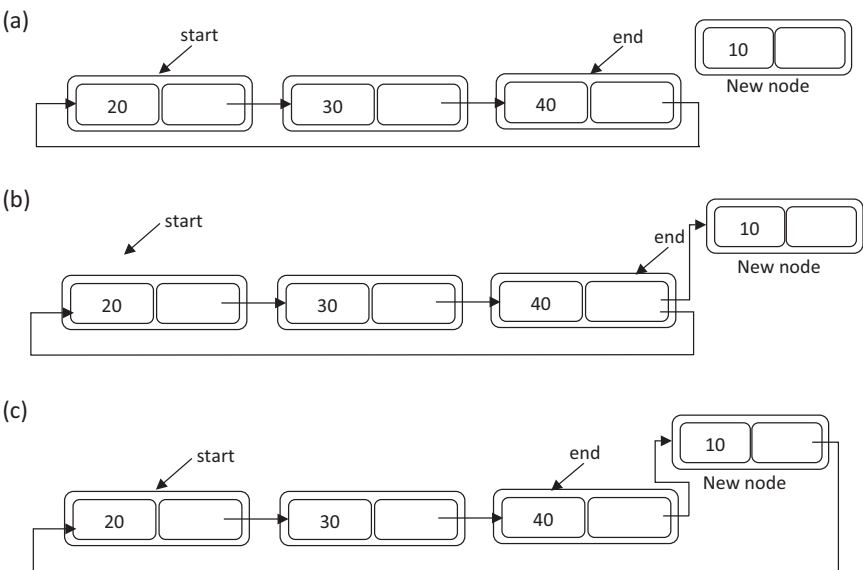


Figure 4.18 (a) Insertion at the end in circular linked lists. (b) Assign end's next to Newnode (c) Assign next of newnode to start

A program segment that does this with the usual notation is the following:

- The method takes two parameters: *self* (referring to the instance of the class) and *newdata* (the data to be inserted into the new node).
- A new node is created using the *Node* class, with *newdata* as its data.
- The code checks if the linked list is empty by verifying if *self.start* is *None*. If it is empty, the *insert_At_Begining* method is called to insert the new data at the beginning of the linked list. Then, the method returns.
- If the linked list is not empty, the next attribute of the new node (*NewNode.next*) is set to the current start of the linked list.
- The next attribute of the current end of the linked list (*self.end.next*) is set to the new node (*NewNode*).
- Finally, the end attribute of the linked list is updated to point to the new node (*self.end = NewNode*), making it the new end of the linked list.

Program continues...

```
def insert_At_End(self, newdata):
    NewNode = Node(newdata)
    if (self.start == None):
        self.insert_At_Begining(newdata)
        return
    NewNode.next = self.start
    self.end.next = NewNode
    self.end = NewNode
```

3. Insertion after a given node in a circular linked list

Insertion in the middle is slightly more complicated since the link between two nodes is to be “broken” and reattached to the new node appropriately.

While inserting a node after a node with a given key, first, take a pointer temp and move the pointer to the location after which a new node is to be inserted. Let us assume that temp points to nodes having data 30. We are going to insert the new node after 30 as shown in Figure 4.19. There are two cases:

Case 1: When the node with the given key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, check if temp’s next points to *start*. If so, insert the new node as last node by calling *insert_At_End()* method, otherwise follow these three steps:

- i. Set a new node's next point to the node to which temp's next is pointing.
- ii. Assign the new node's next to temp's next.
- iii. Make the temp's next point to the new node.

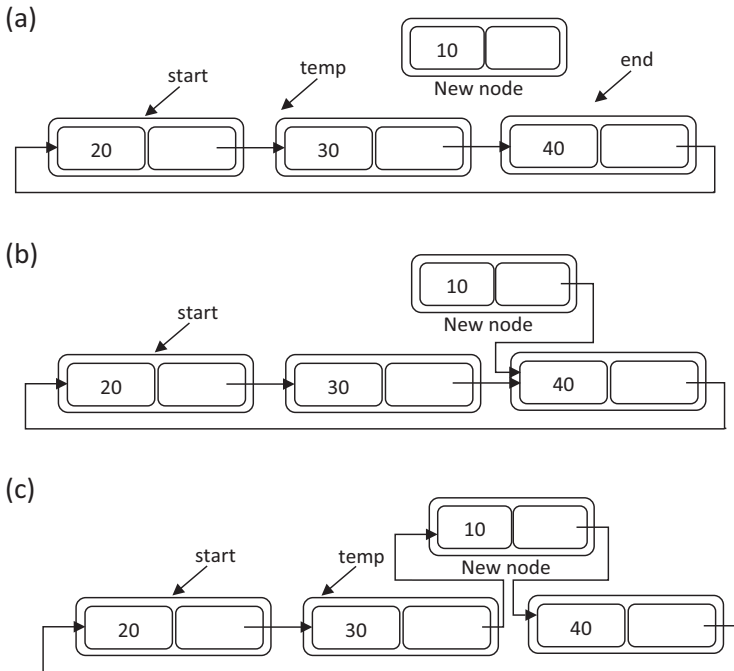


Figure 4.19 (a) Locate temp to desired location (b) Assign newnode's next to temp's next (c) Assign temp's next to Newnode

Program continues...

```
def insert_After(self, data, newdata):
    temp = self.start
    while(temp.data != data and temp.next != self.start):
        temp = temp.next
    if (temp.data != data and temp.next == self.start ):
        print(data, " Not Found")
    elif(temp.data == data and temp.next == self.start):
        self.insert_At_End(newdata)
    else:
        NewNode = Node(newdata)
        NewNode.next = temp.next
        temp.next = NewNode
```

The method takes three parameters: *self* (referring to the instance of the class), *data* (the data value after which the new node will be inserted), and *newdata* (the data to be inserted into the new node).

- A temporary variable *temp* is initialized with the start node of the circular linked list.
- A while loop is used to iterate through the linked list until either the data value is found or until we reach the start node again. The loop condition checks if *temp.data* is not equal to *data* and *temp.next* is not equal to the start node.
- If the loop terminates because the data value was not found (*temp.data != data* and *temp.next == self.start*), a message is printed indicating that the data value was not found.
- If the loop terminates because the data value was found and the next node is the start node (*temp.data == data* and *temp.next == self.start*), the *insert_At_End* method is called to insert the new data at the end of the circular linked list.
- If neither of the above conditions is met, it means the data value was found and it is not the last node. In this case, a new node *NewNode* is created using the *Node* class with *newdata* as its data.
- The *next* attribute of the new node (*NewNode.next*) is set to the next node of *temp*.
- The *next* attribute of *temp* is updated to point to the new node (*temp.next = NewNode*), effectively inserting the new node after *temp*.

4. Insertion before a given node in a circular linked list

While inserting a node before a node with a given key, assign *temp* to the *start*. If the data in the temp node is the same as the key (before which new node to be inserted), call the *insert_At_Begining()* method.

If the node to be inserted is not the first node, then make a pointer *temp* to check the data in the second node (that is temp's next-data). If it matches with the key, insert the new node as a second node, otherwise move the *temp* pointer to the next one. There are two cases:

Case 1: If the key is not found in the linked list, then print the appropriate message.

Case 2: When the node with the given key is found in the linked list, then make the new node's next point to the node to which temp's next is pointing to. Let us assume that temp points to nodes having data 30. We are going to insert the new node before 40 as shown in Figure 4.19; follow these steps.

- i. Set a new node's next point to the node to which temp's next is pointing.

- ii. Assign the new node's next points to temp's next.
- iii. Make the temp's next point to the new node.

Program continues...

```
def insert_Before(self, data,newdata):
    temp = self.start
    if (temp.data == data):
        self.insert_At_Begining(newdata)
    else:
        while(temp.next.data != data and temp.next !=
            self.start):
            temp = temp.next
        if (temp.next.data != data and temp.next == self
            .start ):
            print(data, " Not Found")
        else:
            NewNode = Node(newdata)
            NewNode.next = temp.next
            temp.next = NewNode
```

- The method takes three parameters: *self* (referring to the instance of the class), *data* (the data value before which the new node will be inserted), and *newdata* (the data to be inserted into the new node).
- A temporary variable *temp* is initialized with the start node of the circular linked list.
- An initial check is performed to see if the data value of the start node is equal to the *data* parameter. If it is, the *insert_At_Begining* method is called to insert the new data at the beginning of the circular linked list.
- If the start node's data value is not equal to the *data* parameter, a while loop is used to iterate through the linked list until either the next node's data value is equal to *data* or until we reach the start node again. The loop condition checks if *temp.next.data* is not equal to *data* and *temp.next* is not equal to the start node.
- If the loop terminates because the data value was not found (*temp.next.data != data* and *temp.next == self.start*), a message is printed indicating that the data value was not found.
- If the loop terminates because the data value was found and it is not the last node, a new node *NewNode* is created using the *Node* class with *newdata* as its data.
- The *next* attribute of the new node (*NewNode.next*) is set to the next node of *temp*.
- The *next* attribute of *temp* is updated to point to the new node (*temp.next = NewNode*), effectively inserting the new node before *temp.next*.

iii. Deletion

Deletions from a linked list are comparatively easier than insertions. There are five types of deletions: the first node, the node in the end, a given node, after a given node and before a given node.

1. Deletion of first node

Deleting a node that is the first node of a circular linked list is quite simple. Take a pointer *temp* and assign it to the *start*. Move the *start* to its next node. Set the end's next start and delete the node pointed by *temp*.

The diagram is almost same as deletion of the first node in the singly linked list (Figure 4.20).

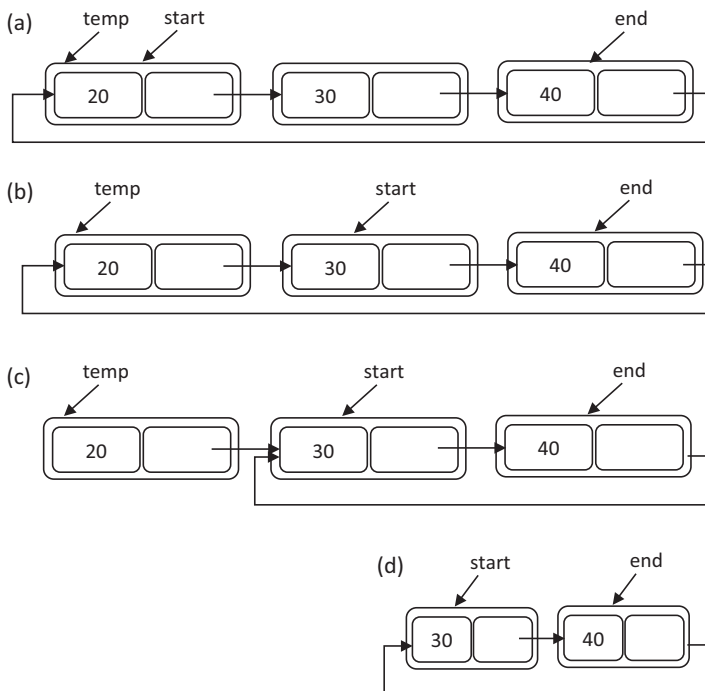


Figure 4.20 (a) Assign temp to start (b) Move start to temp's next (c) Assign end's next to start (d) remove temp

Program continues...

```
def del_first(self):
    temp = self.start
    if (self.start == self.end):
        self.start = self.end = None
        del temp
        return
    self.start = temp.next
    self.end.next = self.start
    del temp
```

- The method *del_first* takes a single parameter *self* (referring to the instance of the class).
- A temporary variable *temp* is initialized with the start node of the circular linked list.
- An initial check is performed to see if the start node is also the end node (*self.start == self.end*). If it is, it means there is only one node in the circular linked list.
- In this case, both the start and end nodes are set to *None*, effectively removing the only node from the list.
- The temporary variable *temp* is deleted using the *del* keyword to free up the memory occupied by the node.
- The *return* statement is used to exit the method.
- If the start node is not the same as the end node (indicating that there are multiple nodes in the list), the following steps are executed:
- The *self.start* attribute is updated to point to the next node after the start node (*self.start = temp.next*). This effectively removes the first node from the list.
- The *next* attribute of the end node (*self.end.next*) is updated to point to the new start node, thereby maintaining the circular nature of the linked list.
- The temporary variable *temp* is deleted using the *del* keyword to free up the memory occupied by the node.

2. Deletion of the node at the end

Deleting a node that is the end node of a circular linked list is quite simple. Take a pointer *temp* and assign *temp* to the *start* (Figure 4.21). There are two cases while deleting the last node:

Case 1: If the end node is the same as the first node. In other words, there is only one node present in the doubly linked list. Call the *del_first()* method.

Case 2: If there are more than one element in the doubly linked list, move the *temp* pointer until the temp's next to next is not the start node using a while loop. In other words, temp points the node before the end node. Assign one more pointer *temp1*, pointing to temp's next. Assign temp's next to start. Set end to temp and delete *temp1*.

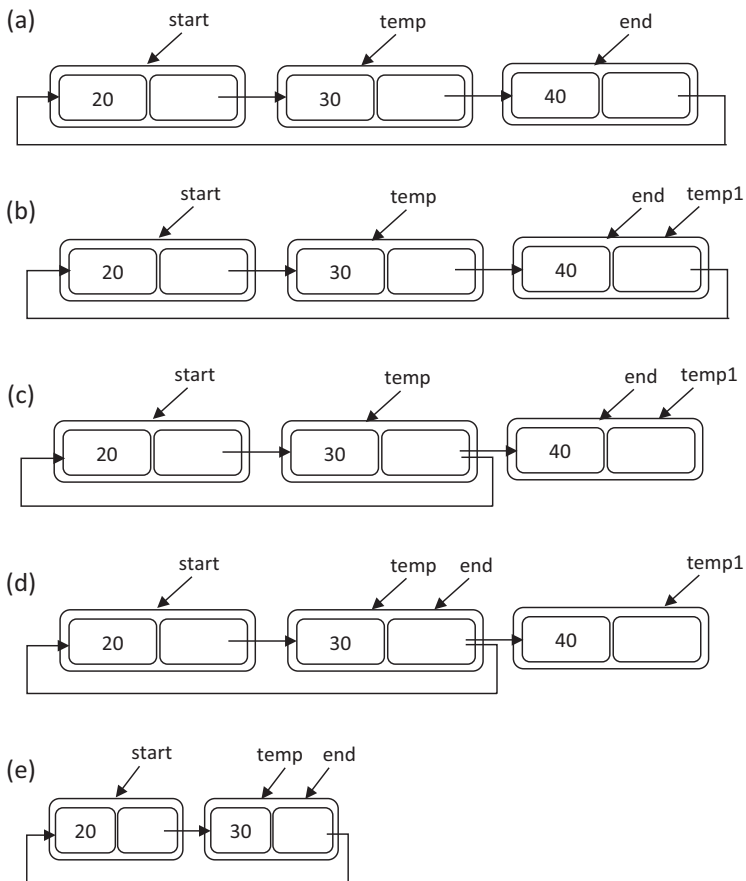


Figure 4.21 (a) Deletion of the node at the end in a circular linked list. (b) Assign temp1 to end. (c) Assign temp's next to start. (d) Assign end to temp. (e) Remove temp1

Program continues...

```
def del_last(self):
    temp = self.start
    if (self.start == self.end) :
        self.del_first()
        return
    while(temp.next.next != self.start):
        temp = temp.next
    temp1 = temp.next
    temp.next = self.start
    self.end = temp
    del temp1
```

3. Deletion of a given node

Deleting a node from the middle is a little more complicated since we have to first traverse the list up to the node we wish to delete (Figure 4.22). Let the node to be deleted contain the key. Depending on the presence of the key, there are four cases.

Case 1: If the key is present in the first node, call the `del_first()` method.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer `temp` and assign it to the `start` pointer. Keep moving the `temp` pointer until `temp`'s next data matches with the `key`. If the key is found at the last node, call the `del_last()` method.

Case 3: If the key is found in between, take one more pointer `temp1` (`temp1` to be deleted) pointing to `temp`'s next, assign `temp`'s next to `temp1`'s next. Now delete `temp1`.

Case 4: If the key is not found, print the appropriate message.

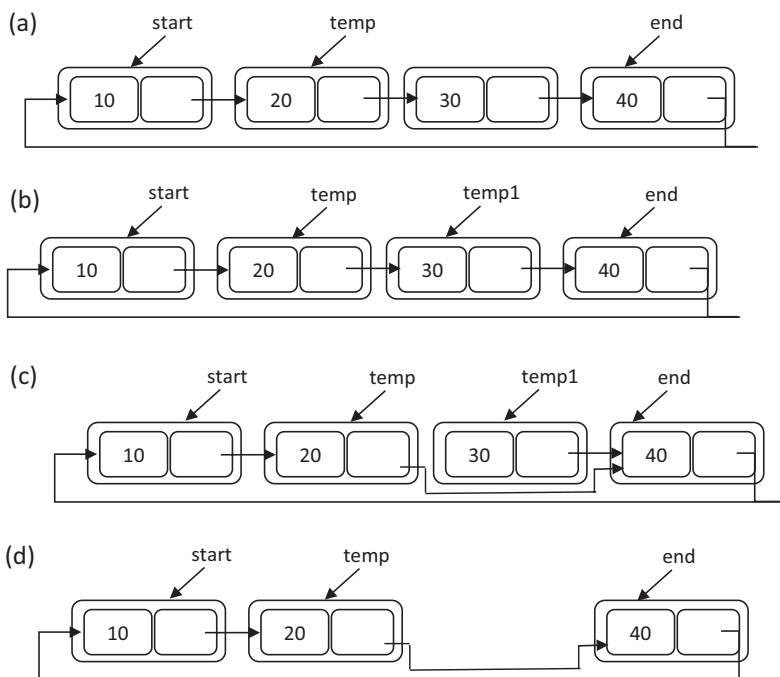


Figure 4.22 (a) Assign `temp` to desired location (b) Assign `temp1` to `temp`'s next (c) Assign `temp`'s next to `temp1`'s next (d) Delete `temp1`

Program continues...

```

def del_node(self, data):
    temp = self.start
    if (temp.data == data ):
        self.del_first()
        return
    while(temp.next.data != data and temp.next.next !=
self.start):
        temp = temp.next

    if (temp.next.data != data and temp.next.next ==
self.start):
        print(data, " Not Found")

    elif(temp.next.data == data and temp.next.next ==
self.start):
        self.del_last()
    else:
        temp1 = temp.next
        temp.next = temp1.next
        del temp1

```

4. Deletion of the node after a given node

Let the node to be deleted contain the key (Figure 4.22). Depending on the presence of the key, there are four cases.

Case 1: If the key is present in the first node, call the *del_first()* method.

Case 2: If the key is not the first node, it can be anywhere in the linked list. To traverse the linked list, take a pointer *temp* and assign it to the *start* pointer. Keep moving the pointer until *temp*'s data matches with the *key*. If the key is found at the last node, call the *del_last()* method.

Case 3: If the key matches with any other intermediate node, take one more pointer *temp1* and assign it to *temp*'s next. Assign *temp1*'s next-*prev* to *temp*, assign *temp*'s next to *temp1*'s next. Now delete the *temp1* node.

Case 4: If the key does not match with data of any node, display appropriate messages.

Program continues...

```

def del_after(self, data):
    temp = self.start
    if (temp.data == data and self.start == self.end ):
        print("There is only one element in the list...")
        print("Next element cannot be deleted..")

```

```
        return
    while(temp.data != data and temp.next != self.start):
        temp = temp.next

    if (temp.data != data and temp.next == self.start ):
        print(data, " data Not found")

    elif (temp.data == data and temp.next == self.start ):
        print(data, " is the Last node.. the next node is
        First node")
        ch = input("Do you to delete the first node (y | Y) : ")
        if (ch == 'y' or ch == 'Y'):

            self.del_first()
            return
    else:
        temp1 = temp.next
        temp.next = temp1.next
        del temp1
```

5. Deletion of the node before a given node

Depending on the presence of the key, there are four cases.

Case 1: If the key matches with the data present in the second node in the circular linked list, call the *del_first()* method.

Case 2: If the key matches with the data present in the first node in the circular linked list, call the *del_last()* method.

Case 3: If none of the above cases are met, the code enters a while loop that continues until either the data value is found or the next node becomes the starting node again.

- i. Inside the loop, *temp* is updated to the next node in the circular linked list.
- ii. After exiting the while loop, the code checks if the data value was found or if the next node is the starting node (indicating that the data value was not found).
- iii. If the data value was not found, it prints a message indicating that the data was not found.
- iv. Otherwise, it initializes *temp1* as the starting node and enters another while loop to find the node immediately before *temp* in the circular linked list.
- v. Once the previous node is found (*temp1*), it updates its *next* pointer to skip the node to be deleted (*temp*) and assigns *temp2* to the node to be deleted.
- vi. Finally, it deletes *temp2* using the *del* keyword.

Program continues...

```
def del_before(self, data):
    temp = self.start
    if ((temp.data == data and self.start == self.end)
        or temp.next.data == data ):
        self.del_first()
        return
    elif(temp.data == data and temp == self.start):
        self.del_last()
        return
    else:
        while(temp.data !=data and temp.next != self.start):
            temp = temp.next
        #while(temp.next.data != data and temp1.next.next
        !=self.start):

        # temp = temp.next
        if (temp.data != data and temp.next == self.start ):
            print(data, " data Not found")
        else:
            temp1 = self.start
            while(temp1.next.next != temp):
                temp1 = temp1.next

            temp2 = temp1.next
            temp1.next = temp
            del temp2
```

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Create a Single Linked List ")
    print("2. Insert at the Beginneing ")
    print("3. Insert at the End")
    print("4. Insert After a given Data ")
    print("5. Insert before a given Data")
    print("6. Delete the First node")
    print("7. Delete the Last node")
    print("8. Delete the a given node")
    print("9. Delete After a given Data")
    print("10. Delete before a given Data")
    print("11. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt

if __name__ == '__main__':
    list1 = CLinkedList()
    i=1
    while (i > 0 and i <=10 ):
        i = menu()
```

```
if (i == 1):
    x = input("Enter data to create the first node :")
    list1.start = Node(x)
    list1.end= list1.start
    list1.end.next = list1.start
    list1.listprint()

elif (i==2):
    x = input("Enter data to insert at the beginning :")
    list1.insert_At_Begining(x)
    list1.listprint()

elif (i==3):
    x = input("\n Enter data to insert at the end :")
    list1.insert_At_End(x)
    list1.listprint()

elif (i==4):
    x = input("\n Enter data to create the node :")
    y = input("Node to be created after data :")
    list1.insert_After(y,x)
    list1.listprint()

elif (i==5):
    x = input("\n Enter data to create the node :")
    y = input("Node to be created before data :")
    list1.insert_Before(y,x)
    list1.listprint()

elif (i==6):
    list1.del_first()
    list1.listprint()

elif (i==7):
    list1.del_last()
    list1.listprint()

elif (i==8):
    x = input("Node to be deleted :")
    list1.del_node(x)
    list1.listprint()

elif (i==9):
    x = input("Node to be deleted after data :")
    list1.del_after(x)
    list1.listprint()

elif (i==10):
    x = input("Node to be deleted before data :")
    list1.del_before(x)
    list1.listprint()

else:
    print("End")
```

- ## Advantages of linked lists

Fill-in-the-blank questions

1. The _____ node always points to the first node of a linked list.
2. The number of pointers that must be changed when an element is inserted in the midst of a linked list is _____.
3. Insertion of an element at the ends of a linked list requires the modification of _____ pointer(s).
4. A _____ linked list stores the address of the head node in the next pointer of the last node.
5. Polynomial addition can be implemented using a _____.

Multiple-choice questions

1. In a circular singly linked list
 - a. Nodes are all linked together in some sequential manner.
 - b. There is no beginning and no end.
 - c. Nodes are arranged hierarchically.
 - d. Forward and backward traversal are permitted.
2. Each node in a linked list consists of two fields. One field is the data field. The other one is
 - a. Pointer to a char
 - b. Pointer to an integer
 - c. Pointer to the node
 - d. None of the above
3. How many null pointers exist in a doubly linked circular list?
 - a. 0
 - b. 1
 - c. 2
 - d. 3
4. An underflow condition in a linked list may occur while attempting to
 - a. Insert a new node when there is no free space for it
 - b. Delete a non-existent node
 - c. Delete a node from an empty list
 - d. Insert a new node in an empty list
5. A variant of a linked list where the last node points to the first node, is a
 - a. Singly linked list
 - b. Doubly linked list
 - c. Circular linked list
 - d. Multiple linked list

-
6. You are given pointers to the first node and the last node of a singly linked list. Which of the following operations is dependent on the length of the list?
- a. Delete the first element of the list
 - b. Insert a new element as the first element
 - c. Delete the last element
 - d. Add a new element at the end of the list
7. Which of the following operations is performed more efficiently in a doubly linked list?
- a. Deleting a node from a given location
 - b. Searching for a given element
 - c. Inserting a node after a given node
 - d. Traversing the list to process each node
8. What is the asymptotic time complexity to add an element to a linked list?
- a. $O(1)$
 - b. $O(n)$
 - c. $O(n^2)$
 - d. None of the above
9. In which type of linked list can traversals be performed in both directions?
- a. Singly linked list
 - b. Doubly linked list
 - c. Circular linked list
 - d. All of the above
10. Which of the following statements is true?
- a. The size of the linked list is dynamic and can be changed as needed.
 - b. Arrays have better cache locality than linked lists.
 - c. Random access of elements at a linked list is not possible.
 - d. All of the above.
11. Which of the following methods is best for locating an entry in the linked list at position k ?
- a. Singly linked list
 - b. Doubly linked list
 - c. Circular linked list
 - d. Array implementation

12. Which of the following issues can be resolved with a linked list and two pointers?
- a. Finding the intersection of two linked lists
 - b. Finding the middle element of a linked list
 - c. Detecting a cycle in a linked list
 - d. All of the above
13. What is absent from a linked list node?
- a. A pointer to the linked list
 - b. Data
 - c. A pointer to the previous node
 - d. A pointer to the next node
14. Which of the following cannot be effectively accomplished with a linked list?
- a. Priority queue
 - b. Stack
 - c. Binary search
 - d. Queue
15. What drawback does a circular linked list have?
- a. A simple loop through the list will never end.
 - b. It is complex to implement.
 - c. It consumes more memory than a simple linked list.
 - d. All of the above.
16. What is a linked list's primary benefit over an array?
- a. In linked lists, random access is more effective.
 - b. It is simpler to implement linked lists.
 - c. Elements can be added to it in $O(1)$ time.
 - d. A linked list requires less memory.
17. Which of these is an application of linked lists?
- a. To implement file systems
 - b. For separate chaining in hash tables
 - c. To implement non-binary trees
 - d. All of the above
18. One example of the _____ form of memory allocation is the linked list.
- a. Static
 - b. Dynamic
 - c. Compile time
 - d. None of the above

19. The linked list data structure provides significant cost savings in
- Space utilization
 - Computational time
 - Computational time and space utilization
 - None of above
20. Pointer to node X in a singly linked list is provided. Can we remove node X from the provided linked list, as there is only one pointer provided and no pointer to the head node?
- Potentially, if X is not the final node.
 - Potentially, if the linked list's size is even.
 - Potentially, if the linked list's size is odd.
 - Potentially, if X is not the first node.

Descriptive questions

- Let A and B be two singly linked lists. Write a program to attach these two lists to construct a singly linked circular list
- Create a doubly-linked list with the following elements and write a program to print it in the reverse order: 3 5 7 9 11 13 15.
- If $A[3][3][3]$ is an integer matrix, calculate the addresses of the following elements:

$A[1][1][1]$, $A[1][2][2]$, $a[2][1][2]$

- Write a program to delete the duplicate elements from an array.
- Write a program to remove alternate nodes from a list and create a new list.
- An array $A = \{3, 5, 7, 9\}$ is given and it is full. Write a program to insert a new element 11 by doubling the array size.
- Given a linked list of integers, write a program to find the largest and smallest elements in the list.
- Write a program to find a sub-list in a given linked list.

Answers to true–false questions

- True
- False
- False
- True
- False

Answers to fill-in-the-blank questions

1. start
2. two
3. one
4. circular
5. linked list

Answers to multiple-choice questions

- | | | | | | | | | | |
|-----|------|-----|---|-----|---|-----|---|-----|---|
| 1. | a, b | 2. | c | 3. | a | 4. | c | 5. | c |
| 6. | c | 7. | b | 8. | b | 9. | b | 10. | d |
| 11. | d | 12. | d | 13. | a | 14. | c | 15. | a |
| 16. | c | 17. | d | 18. | b | 19. | c | 20. | a |